

IT Workshop

Course Code: CS2204

Lecture 2

Dr. Rahul Mishra

IIT Patna



- *An operating system is the software that manages the computer's hardware and provides a convenient and safe environment for running programs.*
- It acts as an interface between **programs and the hardware resources** that these programs access (like **memory, hard disk, and printer**).
- It is loaded into memory when a computer is booted and remains active as long as the machine is up.
- To grasp the key features of an operating system, let's consider the management tasks it has to perform when we run a program.
- These operations also depend on the operating system we are using, but the following actions are common to most systems.

- It allocates memory for the program and loads the program to the allocated memory.
- It also loads the CPU registers with control information related to the program.
- The instructions provided in *the program are executed by the CPU*.
- The *operating system keeps track of the instruction that was last executed*. This enables it to resume a program if it had to be taken out of the CPU before it completed execution.
- If the program needs to access the hardware, it makes a *call to the operating system rather than attempt to do the job itself*.

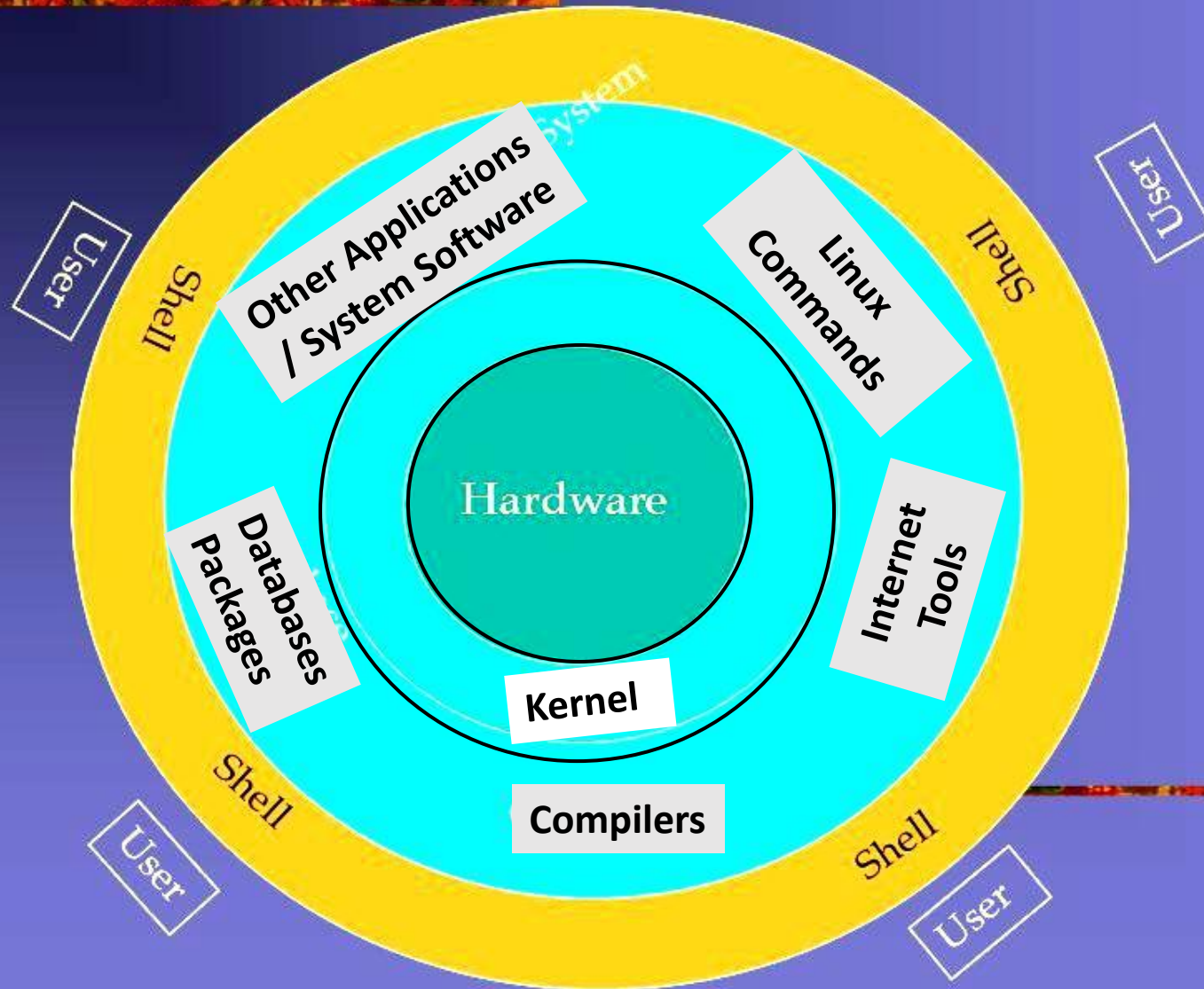
- If the program *needs to read a file on disk*, the OS directs the disk controller to open the file and make the data available to the program.
- After the program has completed execution, the *OS cleans up the memory registers* and makes them available for next program.
- Modern OS are *multiprogramming*, i.e., they allow multiple programs to be in memory.
- Single and multicore processors operating systems.

UNIX Operating System

Features of UNIX

- 1. Portability:** Can run on various hardware systems with minimal modifications.
- 2. Multiuser Capability:** Supports multiple users accessing the system simultaneously.
- 3. Multitasking:** Executes multiple processes concurrently.
- 4. Hierarchical File System:** Organized file structure for efficient storage and retrieval.
- 5. Security and Stability:** Robust design ensures secure and stable operations.

Kernel-Shell Relationship



UNIX Architecture

- 1. Kernel:** The core component managing hardware and system resources.
- 2. Shell:** Command-line interface for users to interact with the system.
- 3. File System:** Organizes and manages data storage hierarchically.
- 4. Utilities:** Built-in tools for performing various tasks such as file management, process control, and text processing.

Shells in UNIX

1. **Bourne Shell (sh):** The original UNIX shell.
2. **C Shell (csh):** Provides a syntax similar to the C programming language.
3. **Korn Shell (ksh):** Combines features of the Bourne Shell and C Shell.
4. **Bash (Bourne Again Shell):** Popular shell used in Linux and UNIX systems.

Applications of UNIX

- 1. Server Operating System:** Widely used for web servers, database servers, and network management.
- 2. Development Environments:** Preferred by developers for programming and software development.
- 3. Scientific Research:** Used in simulations, data analysis, and computational tasks.
- 4. Embedded Systems:** Forms the backbone of devices like routers and smart appliances.

UNIX vs. Linux

1. UNIX:

- Proprietary operating system.
- Primarily used in enterprises and servers.

2. Linux:

- Open-source and free.
- Derives from UNIX principles.
- Extensive community support and hardware compatibility.

Regular Expressions, **grep** and **sed**

What Is A Regular Expression?

- A *regular expression* is a pattern consisting of a sequence of characters that is matched against text.
- Regular expressions give us a way of recognizing words, numbers and operators that appear as part of a larger text so the computer can process them in a meaningful and intelligent way.

What are Atoms?

- Regular expressions consist of atoms and operators.
- An **atom** specifies what text is to be matched and where it can be found.
- There are five types of atoms that can be found in text:
 - Single characters
 - Dots
 - Classes
 - Anchors
 - Back references

Single Characters

- The most basic atom is a single character; when a single character appears in a regular expression, that character must appear in the text for there to be a successful match.
- Example (String is "**Hello**"; Regular Expression is "**l**")
 - The match is successful because "**l**" appears in "**Hello**"
 - If their regular expression had been "**s**", there would be no match.

Dot

- A *dot* (".") matches any character except new line ('\n').
- Example
 - **a.** matches **aa**, **ab**, **ac**, **ad**, **aA**, **aB**, **a3**, etc.
 - **.** will match any character in **HELLO**, **H.** will match the **HE** in **HELLO**, **h.** matches nothing in **HELLO**.

Class

- A class consists of a set of ASCII character, any one of which matches any character in the text.
- Classes are written with the set of characters contained within brackets.
- Example: **[ABL]** matches either "L" in **HELLO**.

Ranges and Exceptions in Classes

- A range of characters can be used in a class:
 - **[a-d]** or **[A-Za-z]**
- Sometimes is it easier to specify what characters DON'T appear.
- This is done using exclusion (^).
- Examples
 - **[^aeiou]** specifies anything but a vowel.
 - **[^0-9]** specifies anything but a digit.

Classes – Some Examples

Regular Expression	Means
[A-H]	[ABCDEFGH]
[A-Z]	Any uppercase letter
[0-9]	any digit
[[a]	[or a
[0-9\ -]	digit or hyphen
[^AB]	Any character except A or B
[A-Za-z]	Any letter
[^0-9]	Any character other than a digit
[]a]	] or a
[^\^]	Anything but ^

Anchors

- Anchors line up the pattern with a particular part of the string:
 - `^` Beginning of the line
 - `$` End of the line
 - `\<` Beginning of a word
 - `\>` End of a word

anchors- Examples

- Sample text: One line of text\n
- **^One** Matches
- **text\$** Matches
- **\<line** Matches
- **\>line** Does not match

What are Operators?

- Operators provide us with a way to combine atoms to form larger and more powerful regular expressions.
- Operators play the same role as mathematical operators play in algebraic expressions.
- There are five types of operators that can be found in text:
 - Sequence, **Alternation**, Repetition, **Group**, and Save

Sequence

- No symbol is used for the sequence operator; all you need is to have two atoms appear in sequence.
- We can match the string **CHARACTER** with the pattern **ACT** because we find the sequence **ACT** in our string.

Sequence - Examples

- **dog** – matches the character sequence "**dog**"
- **a..b** – matches **a**, any two characters, then **b**
- **[2-4][0-9]** – matches a number between **20** and **49**.
- **^\$** - matches a blank line
- **^.\$** - matches a line with only one character
- **[0-9] – [0-9]** – matches two digits with a dash in between.

Alternation

- The alternation operator (|) defines one or more alternatives, either of which can appear in the string.
- Examples
 - **UNIX|unix** matches either **UNIX** or **unix**
 - **Ms|Mrs|Miss** matches **Ms**, **Mrs** or **Miss**
 - **FE|EL** matches **HELLO** because one of the alternatives matches it.

Repetition

- Repetition refers to a definite or indefinite number of times that one or more characters can appear.
- The most common forms of repetition use three "short form" repetition operators:
 - * - zero or more occurrences
 - + - one or more occurrences
 - ? - zero or one occurrences

* Examples

- **BA^*** - **B, BA, BAA, BAAA, BAAAA**
- **$B.*$** - **B, BA, BB, BC, BD, ..., BAA, BAB, BAC, ...**
- **$.^*$** - any sequence of zero or more characters

+ - Examples

- **BA⁺** - **BA, BAA, BAAA, BAAA, ...**
- **B.⁺** - **BA, BB, BC, BD, ..., BZ, BAA, BAB, ...**
- **.⁺** - any sequence of one or more characters

? - Examples

- **d?** - zero or one d
- **[0-9]?** – zero or one digit
- **[^A-Z]?** – zero or one character except a capital letter
- **[A-Za-z]?** – zero or one letter

General Cases of Repetition

- Repetition can be stated in more general terms using a set of escaped brackets containing two numbers separated by a comma
- Example
 - **B\{2, 5\}** would match **BB, BBB, BBBB, BBBB**
- The minimum or maximum value can be omitted:
 - **CA\{5\}** matches **CAAAAA**
 - **CA\{2, \}** matches **CAA, CAA, CAAA,...**
 - **CA \{, 5\}** matches **CA, CAA, CAAA, CAAAA, CAAAAA**
(escape so the braces are interpreted as char)

What is `grep`?

- **`grep`** (general regular expression program) allows the user to print each line in a text file that contains a particular pattern.

What is grep?

- The name **grep** stands for "*g*eneral *r*egular *e*xpression *p*rogram."

- The general format is

grep *pattern filenames*

- The input can be from files.
- **grep [options] pattern [files]**

Supporting Files

- File 1: `file1.txt`

apple

banana

grape

orange

pineapple

grapefruit

Supporting Files

- File 2: `file2.txt`

cat

dog

elephant

fox

grape

Supporting Files

- File 3: `logfile.log`

[ERROR] Unable to connect to server

[INFO] Server started successfully

[WARNING] Low disk space

[ERROR] Disk read failure

[INFO] Backup completed

Basic `grep` Commands

1. Basic Search: `grep grape file1.txt`

#Finds lines containing 'grape'.

2. Case-Insensitive Search: `grep -i GRAPE file1.txt`

#Finds 'grape' irrespective of case.

3. Search in Multiple Files: `grep grape file1.txt file2.txt file3.txt logfile.log`

#Searches for 'grape' across files.

Basic `grep` Commands

4. Display Line Numbers: `grep -n grape file1.txt`

#Shows matching line numbers.

5. Invert Match: `grep -v grape file1.txt`

#Displays lines not containing grape.

6. Count Matches: `grep -c grape file1.txt`

#Counts occurrences of 'grape'.

Advanced `grep` Commands

7. Recursive Search: `grep -r grape`

#Searches in current directory and subdirectories.

8. Whole Word Search: `grep -w grape file1.txt`

Finds 'grape' as a whole word.

9. Highlight Matches: `grep --color grape file1.txt`

#Highlights matches.

Regular Expressions in `grep`

10. *Match Beginning of Line:* `grep ^apple file1.txt`

11. *Match End of Line:* `grep grape$ file1.txt`

12. *Use Regular Expressions:* `grep [aeiou]$ file1.txt`

Sequence, Alternation, Repetition, and Groups

- File: `data.txt`

apple 123

banana 456

grape 789

orange 1001

pineapple 1212

13. *Sequence Example*: `grep -E apple 123 data.txt`

14. *Alternation Example*: `grep -E apple|orange data.txt``

Sequence, Alternation, Repetition, and Groups

15. *Repetition Example:* `grep -E [0-9]+ data.txt`

16. *Grouping Example:* `grep -E 'apple|grape' [0-9] data.txt`

17. *Specific Repetition:* `grep -E [0-9]{3} data.txt`

Saving and Capturing Groups

18. *Save (Capture Groups):* `grep -E '([a-zA-Z]+) ([0-9]+)' data.txt`

19. *Match Start of Line:* `grep -E '^apple' data.txt`

20. *Match End of Line:* `grep -E '55$' data.txt`